

NobleProg

The World's Local Training Provider



Sesión 2

Patrones de migración

Migración de Monolitos a Microservicios

Carlos Georg Lübbe · 3 h

Objetivo de hoy

Construir una **caja de herramientas** completa de patrones de migración.

1. **Strangler Fig** — interceptar y reemplazar desde el borde
2. **ACL** — traducir sin que el legacy contamine lo nuevo
3. **Big-Bang** — por qué casi nunca, y el "casi"
4. **Branch by Abstraction** — reemplazar lo enterrado
5. **Parallel Run** — ganar confianza con tráfico real
6. **Bubble Context** — remodelar el dominio dentro del legacy
7. **Comparativa** — elegir y combinar

1. Patrón Estrangulador

Strangler Fig

Concepto y origen

La **higuera estranguladora**: crece alrededor del árbol hasta reemplazarlo, sin un corte abrupto.
(*Martin Fowler, 2004.*)

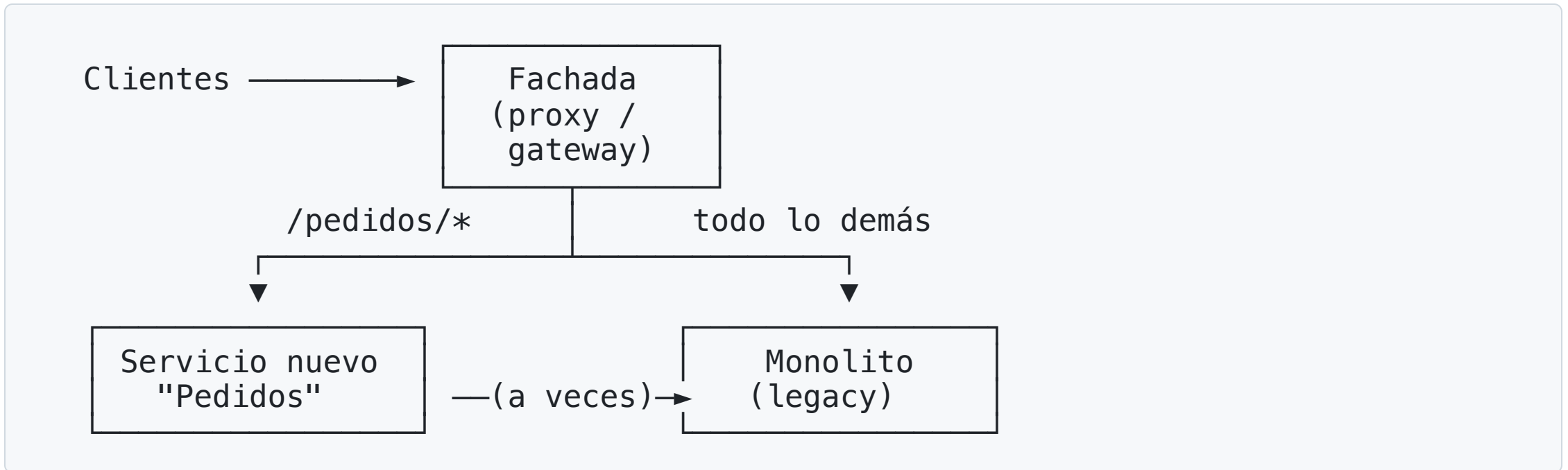
El sistema nuevo **crece alrededor** del viejo hasta sustituirlo.

La idea esencial: **interceptar las peticiones** al monolito y, funcionalidad a funcionalidad, redirigirlas al sistema nuevo.

Tres pasos en bucle:

1. **Identificar** una funcionalidad
2. **Implementarla** en un servicio nuevo
3. **Redirigir** sus llamadas al servicio nuevo

Arquitectura: la fachada de enrutado



La pieza clave es la **fachada** (proxy/gateway) delante del monolito.

Propiedades clave

- **Paso 0:** la fachada se introduce **antes** de extraer nada (enruta 100 % al monolito). Bajo riesgo, infraestructura lista.
- **Redirección gradual** (*canary*): 1 % → 10 % → 100 % del tráfico.
- **Rollback trivial:** volver a apuntar la ruta al monolito.
- El corte natural es por **funcionalidad vertical** visible (URLs, operaciones de API), no por capas internas.

Enrutado gradual: ¿cómo se elige la cohorte?

La ruta dice *qué* funcionalidad; un **predicado extra** sobre la petición dice *a quién* se le da lo nuevo:

1. **Identity / claims** — gateway tras el login: lee `role=employee` del JWT/sesión.
2. **Red / IP** — personal interno por VPN/IP de oficina → allowlist, sin identidad.
3. **Cabecera upstream** — auth en el borde sella `X-User-Type: internal`.
4. **Cookie / feature-flag** — cookie de cohorte para usuarios opt-in.
5. **Porcentaje** — hash del user/sesión → bucket del 1 %.

(1) y (3) exigen que el gateway **vea la identidad**; si no, solo IP o %.

💡 Si el monolito gestiona su propia auth, su token es opaco al gateway → **externalizar el auth suele ser de las primeras extracciones**: desbloquea el enrutado por identidad.

Ventajas y limitaciones

Ventajas

- Incremental; el sistema funciona siempre
- Reversible en cada paso
- Progreso visible y medible (% de tráfico en lo nuevo)
- No exige tocar el interior del monolito

Limitaciones

- Necesita un **borde interceptable** (HTTP ideal); si está enterrado → *Branch by Abstraction*
- **Los datos no se estrangulan solos** (Sesión 3)
- La fachada es un punto único: fiable y observable
- Riesgo de **estrangulamiento eterno** si nadie prioriza terminar

Ejemplo: extraer "devoluciones"

E-commerce monolítico → extraer gestión de devoluciones:

1. Gateway delante; todo el tráfico pasa por él (sin cambios)
2. `returns-service` con su propia BD **para lo que posee; todavía** lee datos maestros (clientes, pedidos) del monolito vía API interna — *los datos aún no están separados (Sesión 3)*
3. Redirigir `/devoluciones/*` solo a empleados internos (1 %) → observar → 100 %
(aquí se verifica; si el riesgo lo pide, comparando con el monolito vía Parallel Run → §5)
4. **Borrar** el código de devoluciones del monolito ✂ ← *el paso que nadie debe olvidar*

Ejercicio 2.1

El bucle del estrangulador

(ver cuaderno de ejercicios)

2. Capa Anticorrupción

Anticorruption Layer (ACL)

Concepto y relación con DDD

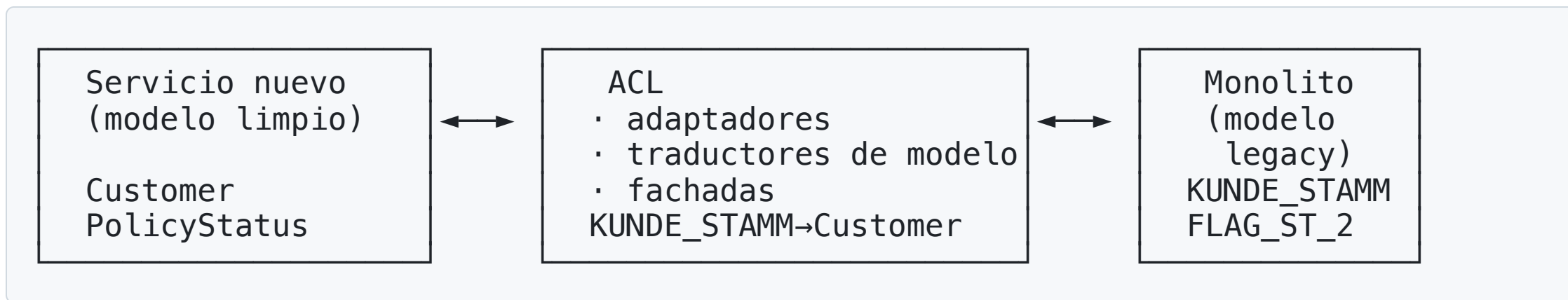
(Término de Domain-Driven Design, Eric Evans.)

Problema: cuando lo nuevo habla con el legacy, el modelo del legacy (nombres confusos, campos sobrecargados, reglas implícitas) **se filtra** y corrompe el modelo nuevo.

Si el servicio nuevo acaba lleno de `flagEstado2` y `tipo_cliente_aux`, la reescritura no ha limpiado nada.

Solución: una capa de **traducción explícita** entre los dos modelos.

La capa de traducción



- **Adaptador**: convierte protocolo/tecnología (SOAP→REST, tablas→objetos)
- **Traductor de modelo**: mapea conceptos (`FLAG_ST_2='X'` → `PolicyStatus.SUSPENDED`)
- **Fachada**: simplifica una interfaz legacy enrevesada

Reglas de oro de la ACL

- La ACL **pertenece al sistema nuevo** (la escribe y mantiene su equipo, en sus términos)
- El modelo legacy **no cruza** la ACL: ningún tipo, nombre ni convención del legacy aparece más allá
- Es **código de usar y tirar a largo plazo**: cuando el legacy muera, la ACL se borra con él → puede ser pragmática, no perfecta

Beneficios y limitaciones

Beneficios

- Protege la integridad del modelo nuevo (la razón de la reescritura)
- Aísla el cambio: si el legacy cambia, solo se toca la ACL
- Documenta de forma ejecutable el mapeo viejo ↔ nuevo

Limitaciones / costes

- Código adicional que escribir y mantener (semánticas que no mapean 1:1, datos sucios)
- Un salto más (latencia, posible punto de fallo)
- Riesgo de **mini-monolito** si una sola ACL gigante lo centraliza todo → mejor **una ACL por contexto**

Ejemplo: la aseguradora

`policy-service` lee pólizas de un **mainframe**. *Activa* = cobertura **en vigor** (concepto de negocio, no un código).

Lo que el dominio ve (modelo nuevo): `Policy { id, status: ACTIVE | LAPSED | CANCELLED }`

El mapeo vive dentro de la ACL:

```
toPolicy(raw):                                # raw = registro del mainframe
    if raw.MIGR_FLAG == 'S': st = migrStatus(raw.id)  # ← 2ª tabla
    elif raw.ST == '02':      st = ACTIVE
    elif raw.ST == '07':      st = LAPSED
    else:                      st = CANCELLED
    return Policy(raw.id, st)
```

El dominio llama `policy.isActive()` ($\equiv$ `status == ACTIVE`); nunca ve `ST`, `MIGR_FLAG` ni la 2ª tabla. Apagar el mainframe → reescribir `toPolicy()`, borrar la ACL; el dominio no cambia.

Ejercicio 2.2

Diseñar una ACL

(ver cuaderno de ejercicios)

3. Reescritura Big-Bang

Por qué es tentadora

- *"El código viejo no tiene arreglo; de cero será más rápido."*
- Psicológicamente atractiva: **campo verde**, tecnología nueva, sin deuda
- Evita la complejidad de la **convivencia** (proxies, sincronización, dos sistemas)

Por qué es arriesgada

- **Objetivo móvil:** el monolito sigue evolucionando durante la reescritura (el negocio no para)
- **El conocimiento está en el código:** miles de reglas no documentadas que se redescubren... en producción
- **Cero valor hasta el final:** todo se entrega de golpe en un corte de altísimo riesgo
- **Segundo sistema (Brooks):** la tentación de añadir "todo lo que faltaba" infla el alcance
- La evidencia es contundente: las reescrituras big-bang grandes **fracasan o se desbordan** con muchísima frecuencia

Cuándo *puede* justificarse

Casos acotados donde es defendible:

- Sistema **pequeño** y **bien especificado** (suite de tests exhaustiva, comportamiento trivial)
- Sistema **congelado de verdad** (sin cambios de negocio durante la reescritura)
- Imposibilidad real de convivencia (plataforma que desaparece) **y** tamaño manejable
- Reescrituras **parciales**: big-bang de *un módulo* dentro de una estrategia incremental

El problema no es reescribir; es reescribirlo **todo** y entregarlo **de golpe**.

4. Branch by Abstraction

El problema que resuelve

Strangler intercepta **desde fuera**. ¿Y si lo que quieres reemplazar está **enterrado dentro**, llamado desde decenas de sitios? → no hay borde donde poner un proxy.

Enfoque ingenuo — *rama de larga vida*: ramificar en git, arrancar lo viejo y construir lo nuevo aparte durante semanas.

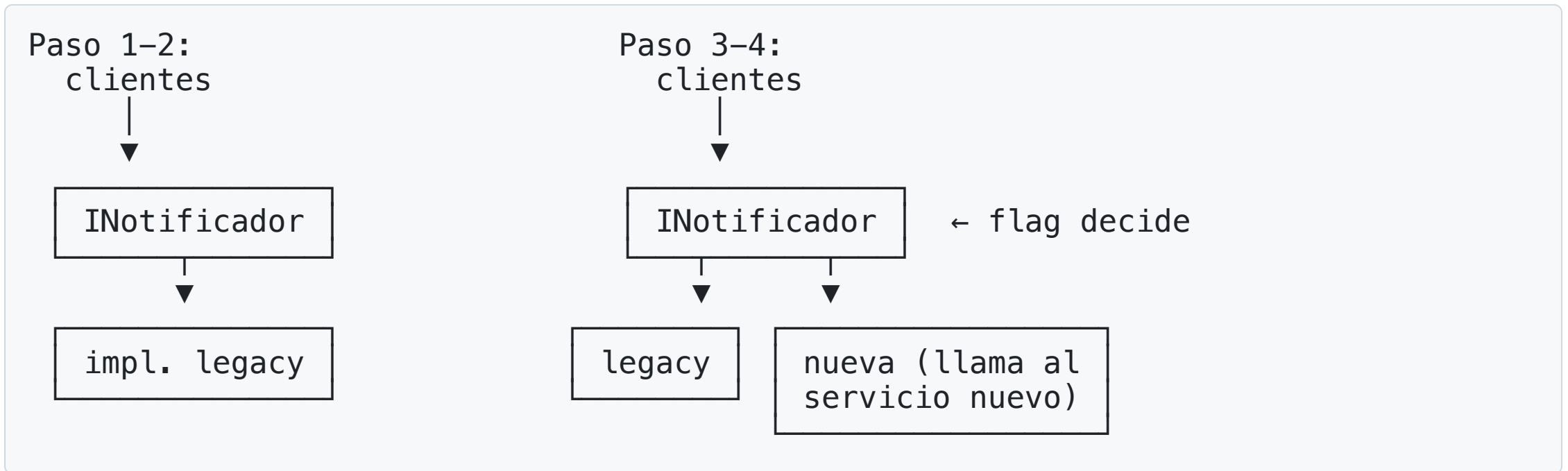
- el resto del equipo sigue en *trunk* (la línea principal, `main`) → la rama **diverge** → infierno de *merges*
- lo que está a medias **no se puede desplegar** → cero valor hasta el final

Idea de BbA: sustituir la rama de *git* por una **rama lógica en el código** — una abstracción con dos implementaciones. Ambas viven en `main`, compilan y se despliegan; **nada diverge**.

Los cinco pasos

1. **Crear una abstracción** (interfaz) sobre la funcionalidad a reemplazar
2. **Migrar los clientes** internos a la abstracción (*pasos 1–2 = pura refactorización*)
3. **Crear la implementación nueva** detrás de la misma abstracción (desplegada pero inactiva)
4. **Conmutar** con un *feature flag* (configurable, rollback al instante)
5. **Limpiar**: borrar la implementación vieja y, si ya no aporta, la abstracción/flag

Visualmente



Ambas implementaciones **conviven** detrás de la interfaz; el flag elige.

La abstracción no es solo una interfaz

Es cualquier costura (*seam*) donde insertar el conmutador:

- **Interfaz / clase abstracta** — lo típico en OO
- **Función de orden superior / Strategy / delegate** — la implementación como dato
- **Fachada, adaptador o inyección de dependencias** — costura a nivel de módulo / *wiring*
- **Proxy / decorador** — un envoltorio que delega a la vieja o la nueva
- **Publicar un evento** en vez de llamar directo — el contrato del evento es la abstracción
- **Costura de datos/infra** — repositorio, vista de BD, SPI/plugin, ruta del gateway

Cuanto más **esparcida** esté la funcionalidad, más arriba conviene la costura (módulo/infra) que una interfaz fina.

Ventajas y limitaciones

Ventajas

- Todo en trunk, despliegues continuos; sin ramas de larga vida
- Dos implementaciones conmutables al instante; cada paso pequeño y seguro
- Combina con Strangler: **Strangler para los bordes, BbA para las entrañas**

Limitaciones

- Exige **poder modificar** el código del monolito (no sirve para legacy intocable/de terceros)
- Una buena abstracción puede ser difícil si la funcionalidad está esparcida
- Coste doble de mantenimiento mientras conviven; requiere disciplina para el paso 5

Ejercicio 2.3

Branch by Abstraction, paso a paso

(ver cuaderno de ejercicios)

5. Parallel Run

Concepto y variantes

A veces conmutar —aunque sea gradual— **da miedo**: cálculos financieros, tarificación, decisiones regulatorias.

Parallel Run: ejecutar **las dos implementaciones a la vez** con las mismas peticiones reales y **comparar resultados**.

- **Lo viejo es la verdad** (*shadow mode*): el cliente recibe la respuesta del legacy; la del nuevo se registra y compara. Las discrepancias no afectan a nadie.
- **Lo nuevo es la verdad**: el cliente recibe la respuesta nueva; el legacy corre detrás como verificación, antes de apagarse.

Qué se compara y cómo

- Resultados funcionales (¿la prima es la misma?), efectos esperados, y lo **no funcional** (¿latencia aceptable?)
- Hace falta un **comparador** (desde un job nocturno hasta comparación en línea)
-  **Efectos secundarios:** en sombra el nuevo **no** debe enviar emails, cobrar tarjetas ni escribir fuera → stubs/sandbox
-  **No determinismo:** timestamps, redondeos, IDs... las reglas deben tolerarlo o las falsas discrepancias ahogan a las reales

Ventajas, limitaciones y encaje

Ventajas: la validación más fuerte posible — datos reales, sin riesgo; confianza cuantificable ("*99,98 % de coincidencia en 3M de peticiones*"); detecta lo que ningún test sintético encuentra.

Limitaciones: coste doble de ejecución; la infra de comparación es trabajo real; difícil con escrituras; solo verifica lo que el tráfico real ejercita.

Encaje: alto riesgo + **resultado comparable** (cálculos, scoring). Va *dentro* de un Strangler o un BbA, no aislado.

Ejercicio 2.4

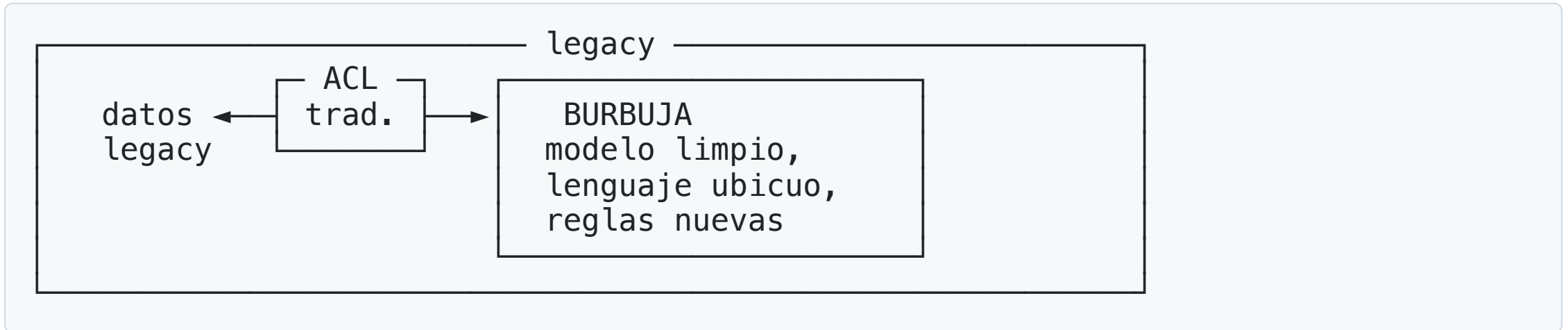
Parallel Run: diseñar la comparación

(ver cuaderno de ejercicios)

6. Bubble Context

Concepto

(*Patrón DDD, Eric Evans.*) Cuando quieres modelar **bien** un dominio nuevo pero todo el entorno es legacy: creas una **burbuja** — un contexto nuevo, pequeño y limpio, **protegido por una ACL**.



La burbuja se centra en **el modelo**, no en la infraestructura: puede vivir *dentro* del despliegue del monolito.

Evolución típica

1. **Burbuja simple:** sin persistencia propia; la ACL traduce desde los datos del legacy en cada acceso
2. **Burbuja con BD propia** (*autonomous bubble*): la ACL pasa a ser una capa de **sincronización** (a menudo asíncrona)
3. La burbuja crece y se le da despliegue propio → **microservicio nacido sano**; o revienta la frontera y absorbe al legacy

Ventajas y limitaciones

Ventajas

- Aplicar **DDD de verdad** desde el día uno en la parte nueva
- Inversión inicial pequeña (no exige plataforma de microservicios)
- Ideal cuando el motor es un **dominio nuevo** o que necesita remodelarse

Limitaciones

- La ACL de **sincronización** puede volverse compleja (sobre todo con escrituras en ambos lados)
- Si la burbuja no crece, queda como **isla rara** dentro del legacy
- Requiere **vigilancia de la frontera** ("saltarse la ACL solo esta vez" es una presión real)

7. Comparativa de patrones

Tabla de decisión

Patrón	Pregunta que responde	Mejor cuando...	Riesgo principal
Strangler Fig	¿Reemplazo funcionalidad del borde?	Hay un punto de entrada interceptable	Quedarse a medias; ignorar datos
ACL	¿Evito que el legacy contamine lo nuevo?	Siempre que lo nuevo hable con lo viejo	Mini-monolito; mantenimiento
Big-Bang	¿Lo reescribo todo de golpe?	Sistema pequeño, congelado, especificado	Objetivo móvil; cero valor
Branch by Abstraction	¿Reemplazo algo enterrado?	Puedes tocar el código; muchos llamantes	No ejecutar la limpieza
Parallel Run	¿Gano confianza en lo nuevo?	Alto riesgo + salida comparable	Coste doble; efectos 2arios
Bubble Context	¿Modelo limpio dentro del legacy?	El motor es remodelar el dominio	Sincronización; burbuja estancada

No son excluyentes: se combinan

Una migración real los **apila**:

- **Strangler Fig** como estrategia global de redirección desde el borde...
- ...usando **Branch by Abstraction** para las piezas internas sin borde...
- ...con una **ACL** en cada frontera entre lo nuevo y lo viejo...
- ...validando lo crítico con **Parallel Run** antes de conmutar...
- ...y arrancando los dominios a remodelar como **Bubble Contexts**.

Criterios transversales de elección

1. **¿Dónde está el corte?** Borde visible → Strangler · Entrañas → Branch by Abstraction
2. **¿Cuál es el motor?** Escalado/entrega → Strangler/BbA · Remodelar dominio → Bubble (+ACL)
3. **¿Cuánto miedo da equivocarse?** Mucho y comparable → añade Parallel Run
4. **¿Puedo tocar el legacy?** No → solo bordes: Strangler + ACL
5. **Siempre:** ACL en las fronteras y **plan explícito de borrado** del legacy

Ejercicio 2.5 + 2.6

Elegir patrón para cada situación

Combinar patrones: plan integral

(ver cuaderno de ejercicios)

¿Preguntas?

Próxima sesión: Domain-Driven Design y descomposición de datos

Bounded contexts · BD por servicio · Saga · Outbox